

Chapter_3_Application_Layer_Network

Chapter 3: Application Layer — Network-Oriented Protocols (Layer 7)

Scope note: This file covers **network-oriented** application layer protocols — DNS (including zone files, TLDs, nslookup), DHCP/TFTP (bootstrapping), SNMP/CMIP (network management), X.500/X.509/LDAP (directory access), and routing algorithms (Dijkstra/OSPF, Bellman-Ford/RIP, BGP). This is **Semester Test 1** material (Chapter 3). Routing algorithms are also implicitly tested in ST2 context.

1. Network-Oriented vs. User-Oriented Protocols

User-oriented protocols (HTTP, SMTP) serve humans directly. **Network-oriented protocols** exist to make the network function or to make it simpler for applications to use it — for example, translating domain names to IP addresses (DNS) or automatically building routing tables (RIP, OSPF).

2. DNS (Domain Name System)

DNS translates (strictly speaking, **resolves**) human-readable domain names into IP addresses.

MCQ fact (2025 ST1 Q5d): DNS **resolves** names (not "translates", not "maps" — though those are close; the textbook uses "resolves").

2.1 Before DNS — The `hosts` File

Before DNS, mappings were kept in a local file. On Linux/Unix: `/etc/hosts`. Format: first entry is the **IP address**, second entry is the **name** (e.g., `127.0.0.1 localhost`).

MCQ fact (2023/2024 ST1): A file on a name server that maps names to IP addresses is strictly called a **zone file** (not a "hosts file", not a "name file", not a "resolution file").

2.2 DNS Basics

Detail	Value
DNS port	53

Detail	Value
Transport protocols	Both TCP and UDP — UDP for standard fast queries; TCP for zone transfers and large responses
DNS server software	<code>bind</code> (the 'd' stands for dæmon)
Tool to resolve names interactively	<code>nslookup</code>

MCQ fact (2025 ST1 Q14): DNS uses **both TCP and UDP** (not just UDP, and not just TCP). Answer: "More than one of the above."

2.3 TLDs and the DNS Hierarchy

Type	Description	Examples
gTLD (Generic)	Generic purpose	<code>.com</code> , <code>.org</code> , <code>.net</code>
ccTLD (Country-code)	Assigned to countries/territories	<code>.za</code> , <code>.io</code> (British Indian Ocean Territory), <code>.is</code> (Iceland)
sTLD (Sponsored)	Require adherence to policies from sponsors	<code>.capetown</code> (both ICANN and ZADNA), <code>.gov</code>

MCQ fact (2025 ST1 Q9): The `.io` TLD is a **ccTLD** (country-code TLD for the British Indian Ocean Territory). It is not a gTLD or sTLD.

MCQ fact: The `.nom.za` second-level domain is intended to allow **individuals** to register subdomains. (*Distractor: "nomads" — trap. ".nom" looks like "nomads" but means "nominative" i.e., individual names.*)

MCQ fact: The second-level domain `tech.za` **does not exist**.

MCQ fact: New TLDs must be approved by **ICANN**. The `.za` ccTLD is managed by **ZADNA**. The `.co.za` registry is operated by **ZACR** (formerly UniForum SA).

Landrush phase: When a new TLD is in *landrush*, it means the domain recently started accepting registrations from the general public — typically at a higher initial cost.

WHOIS: The `whois` protocol retrieves ownership information about a domain. Port **43**. Web-based WHOIS is available at `who.is`.

MCQ fact (2025 ST1 Q2g): WHOIS is associated with port **43**.

Domain registration terminology:

Term	Meaning
Registrar	The organisation through which you register a domain
Registrant	The person/organisation that registered the domain (the owner)
ZACR	The registry operator for .co.za
admin contact	Administrative contact in WHOIS
tech contact	Technical contact in WHOIS
redacted	WHOIS field hidden due to privacy concerns

2.4 Zone Files and Resource Records

A **zone file** is the file on a name server that contains resource records (RRs) for a domain.

The `@` symbol in a zone file represents the domain itself (e.g., `xx.co.za` without any prefix).

Record Type	Purpose	Example
A	Maps name to IPv4 address	<code>www A 41.203.18.59</code>
AAAA	Maps name to IPv6 address	<code>@ AAAA 2001::1234</code>
CNAME	Canonical name (alias)	<code>ftp CNAME www</code>
MX	Mail exchange (with priority)	<code>@ MX 10 ASPMX.L.GOOGLE.COM.</code>
NS	Authoritative name server	<code>@ NS ns1.host-h.net.</code>
PTR	Reverse lookup (IP → name)	<code>59 PTR www.xx.co.za.</code>
SOA	Start of Authority	Serial, Refresh, Retry, Expire, Min TTL
HOST	TXT-like hardware description	<code>@ HOST Raspberry Pi</code>

MCQ fact (MX priority): For MX records, the **lower** priority number is **higher** priority. `MX 5` is preferred over `MX 10`.

SOA record fields:

Field	Meaning
Serial	Version number; by convention <code>YYYYMMDDnn</code> (e.g., <code>2001102002</code> = 20 Oct 2001, version 02)
Refresh	How often secondary name servers check for updates
Retry	How long a secondary server waits to retry after a failed synchronisation attempt
Expire	How long a secondary server serves cached data if primary is unreachable

Field	Meaning
Min TTL	How long clients cache data before requesting fresh copies

MCQ fact (Retry): The **Retry** field in a SOA record indicates the time (in seconds) a **secondary nameserver should wait before retrying a failed zone synchronisation attempt.**

Zone transfer: The process of copying DNS entries from a **primary** to a **secondary** name server.

Example 1 — Zone file for `xx.co.za`:

```
@ A 41.203.18.50
www CNAME xx.co.za.
@ MX 5 ASPMX.L.GOOGLE.COM.
@ MX 10 ALT1.ASPMX.L.GOOGLE.COM.
@ NS ns1.host-h.net.
@ NS ns2.host-h.net.
abc NS ns1.host-h.net.
abc NS ns2.host-h.net.
@ AAAA 2001::1234
@ HOST Raspberry Pi
```

Example 2 — Reverse DNS (PTR):

To look up who owns `192.168.18.59`, query `59.18.168.192.in-addr.arpa`. The `arpa` pseudo-TLD is used for reverse lookups. Note how the address bytes are reversed.

2.5 Name Resolution: Iterative vs. Recursive

Iterative resolution: The client contacts each server in turn, following NS referrals until it gets the answer. The resolver does all the work.

Recursive resolution: The client asks a resolver, and that resolver does all the iterative work on behalf of the client, returning the final answer.

MCQ fact (2025 ST1 Q): There are **13 root name servers**. Their FQDNs are `a.root-servers.net` through `m.root-servers.net`.

MCQ fact (Root servers lexicographic order): If the FQDNs of DNS root name servers are ordered lexicographically, the **last entry** in the list is `m.root-servers.net`.

MCQ fact (Root server behaviour): Root name servers perform **none of the above** in terms of recursive, iterative, or non-authoritative resolution. They simply return **NS referrals** (pointers to TLD name servers) — they do not resolve names themselves.

MCQ fact: The bulk, if not all, of the resource records in DNS root name servers are of type **NS**.

FQDN: A Fully Qualified Domain Name ends with a full stop (.) to unambiguously indicate it is fully qualified (e.g., `www.example.com.`). Without the trailing dot, `nslookup` treats it as relative.

Non-authoritative answer: A response served from a name server's **cache** rather than directly from the authoritative zone file. The data may be slightly stale (up to the TTL).

Example 1 — Iterative resolution of `www.example.com` :

```
nslookup → set server to f.root-servers.net (192.5.5.241)
> set type=ns
> com.           → returns list of .com name servers (A-M.GTLD-SERVERS.NET)
→ set server to a.gtld-servers.net
> example.com.   → returns NS records for example.com
→ set server to ns1.example.com
> www.example.com → returns A record: 192.0.32.10
```

Example 2 — `nslookup` commands:

```
nslookup
> server 10.1.1.1      ← change server
> set type=ns          ← ask for NS records instead of A records
> com.                ← query for com NS servers (trailing dot = FQDN)
> set type=a
> www.example.com     ← normal A record lookup
```

3. Bootstrapping Protocols

DHCP (Dynamic Host Configuration Protocol): Dynamically supplies a booting host with IP address, default gateway, DNS server addresses, and other configuration. Eliminates manual configuration.

TFTP (Trivial File Transfer Protocol): Very simple file retrieval protocol. Uses **UDP**. Classic use: diskless computers use TFTP to transfer an OS image from a network server when they

boot.

MCQ fact: TFTP uses **UDP** (not TCP). It is used by diskless computers to boot over the network.

4. Network Management Protocols

To manage a network, each node runs an **agent** that collects data and executes management actions. A central **manager** coordinates everything.

The agent and manager use a **dual client-server** arrangement: the manager initiates `Get / Set` requests (manager=client, agent=server), and the agent initiates `Trap / Inform` messages on critical events (agent=client, manager=server).

MCQ fact (2025 ST1 Q20): An SNMP agent is **both a client and a server** (answer: "More than one of the above").

4.1 SNMP

SNMP (Simple Network Management Protocol) is the most popular network management protocol. Standardised by IETF.

Manager operations:

- `Get` — retrieve one variable
- `GetNext` — retrieve next variable
- `GetBulk` — retrieve multiple variables
- `Set` — write a variable

Agent operations:

- `Trap` — unsolicited alert to manager on a critical event
- `Inform` — acknowledged alert to manager

SNMP specifies its messages in **ASN.1** and requires **BER** encoding.

4.2 CMIP

CMIP (Common Management Information Protocol) is the OSI equivalent of SNMP. More capable than SNMP but used primarily in **telecommunications** environments. The TCP/IP version is **CMOT** (CMIP over TCP/IP).

MCQ fact: Examples of network management protocols: SNMP, CMIP, CMOT.

5. Directory Access Protocols

Directories organise information (especially about people) in hierarchical structures. They often include **authentication information** enabling **single sign-on** — users authenticate once against the directory, and all applications use that authentication.

5.1 X.500

X.500 is the OSI standard for directory services. Developed by CCITT (now ITU-T) and ISO. Organises data in a **Directory Information Tree (DIT)**.

Components:

- **DSA** (Directory System Agent) — manages the tree
- **DUA** (Directory User Agent) — queries the tree
- **DAP** (Directory Access Protocol) — DUA to DSA communication
- **DSP** (Directory System Protocol) — DSA to DSA communication
- **DN** (Distinguished Name) — the unique identifier for a DIT entry, e.g., {C=ZA, O=SANDF, OU=Air Force, CN=Col Burger}
- **RDN** (Relative Distinguished Name) — each component of the DN

5.2 X.509

X.509 is part of the X.500 series. Deals with **Public-Key and Attribute Certificate Frameworks** — i.e., digital certificates. Widely used for TLS/HTTPS.

MCQ fact: A protocol that implements a public key infrastructure for encryption purposes: **X.509**.

5.3 LDAP

LDAP (Lightweight Directory Access Protocol) was developed because X.500 was too complex. Used for single sign-on. Arranges data in a hierarchy identified by a **base DN**.

LDAP uses a subset of **BER** encoding for all its messages.

6. Gateway Protocols (Routing Algorithms)

Routing is a Layer 3 function, but the protocols that *build* the routing tables operate at Layer 7 (the application layer). These are called **gateway protocols**.

Scope	Name	Examples
IGP (Interior Gateway Protocol)	Operates within an Autonomous System	OSPF, RIP
EGP (Exterior Gateway Protocol)	Operates between Autonomous Systems	BGP

An **Autonomous System (AS)** is an organisation's network. IGP's base decisions on **cost**. EGP's may use policy (e.g., refusing to route through a competitor's AS regardless of cost).

MCQ fact: OSPF and RIP are **IGPs**. BGP is an **EGP**.

6.1 Dijkstra's Algorithm (OSPF)

OSPF (Open Shortest Path First) uses **Dijkstra's algorithm** to compute shortest paths.

Characteristics of Dijkstra: **greedy**, static (non-adaptive by default), centralised (each node runs it locally on a full network map).

OSPF routers broadcast their **view of the network topology (LSDB — Link State Database)** to all other routers. Each router then independently runs Dijkstra on the complete LSDB.

MCQ fact: OSPF uses **Dijkstra's algorithm**. Dijkstra does **not** have a counting-to-infinity problem.

Example — Dijkstra trace (from 2024 ST2 Q5):

Start node A. Nodes: A, B, C, D, E, F, G.

Iteration	Done (Finalised)	Candidates	X (best cost)	Notes
1	A	E	B=3, E=2	A→E cost 2 chosen
2	AE	BDF	B=3, D=2, F=12	A→E→D cost 2 chosen; prior[D]=E
3	AEB	DFG	D=3, F=7, G=12	Actually evaluate all of A,E,B neighbours
...	...	...	...	Continue until all finalised

When tracing Dijkstra, always:

1. Evaluate all neighbours of the **entire finalised set**, not just the most recently added node.
2. The node with the **absolute cheapest total cost** from source is added next.
3. Update prior nodes whenever a cheaper path is found.

6.2 Bellman-Ford Algorithm (RIP)

RIP (Routing Information Protocol) uses the **Bellman-Ford algorithm**.

Characteristics: distributed (each node only knows its neighbours' info), adaptive, simpler to implement.

Counting to infinity is a well-known problem with Bellman-Ford/RIP. When a link fails, nodes can enter a loop where they keep updating each other with stale routes, incrementing the cost indefinitely until it exceeds the maximum hop count (16 in RIP = infinity).

MCQ fact: The **counting-to-infinity** problem affects **Bellman-Ford and RIP** (not Dijkstra/OSPF).

MCQ fact: OSPF algorithm = **Dijkstra**. RIP algorithm = **Bellman-Ford**.

Example 1 — RIP update (from 2023 ST1 Q1f):

Router r has a route to network n at cost 5. Router s receives the routing table from r. s previously had n at cost 8. After processing r's message: s's cost to n = r's cost (5) + cost from s to r (whatever that link cost is, say 3) = 8. If the path via r (5 + cost s→r) is cheaper than 8, s updates.

Example 2 — Bellman-Ford update rule:

```
For each neighbour v with link cost w(u,v):
    new_cost = dist[v] + w(u,v)
    if new_cost < dist[u]:
        dist[u] = new_cost
        prior[u] = v
```

6.3 BGP

BGP (Border Gateway Protocol) is the primary **EGP**. It routes between Autonomous Systems on the Internet. Policy-based, not pure cost-based.

MCQ fact: BGP is an **EGP** (Exterior Gateway Protocol). OSPF and RIP are **IGPs**.

Examinable Practice Questions

Multiple Choice

Q1. Which of the following best describes what DNS does?

A. Encrypts domain names B. Resolves domain names to IP addresses C. Maps IP addresses to port numbers D. Registers new domain names E. Validates SSL certificates

Q2. DNS normally operates on port:

A. 43 B. 53 C. 80 D. 110 E. 443

Q3. Which transport protocols does DNS use?

A. TCP only B. UDP only C. IP only D. More than one of the above E. All of the above

Q4. If the FQDNs of DNS root name servers are ordered lexicographically, the last entry is:

A. l.dns.net B. m.dns.net C. n.dns.net D. l.root-servers.net E. m.root-servers.net

Q5. An agent that monitors and controls network activity in an SNMP-managed network node is a(n):

A. Client B. Server C. P2P-node D. More than one of the above E. All of the above

Q6. Which condition code will be returned by an SMTP server after successfully handling a HELO or EHLO message?

A. 0 B. 150 C. 250 D. 350 E. 450

Q7. The nom.za second-level domain is intended to allow ... to register subdomains within it.

A. Nomads B. Individuals C. Parties who handle nominations for potential office bearers D. Those who provide authoritative definitions of nomenclature E. None, because no such second-level domain exists

Q8. The .io TLD is a(n):

A. sTLD B. ccTLD C. gTLD D. Reserved TLD E. None of the above

Q9. The bulk, if not all, of the resource records in DNS root name servers are of the following type:

A. A B. MX C. NS D. More than one of the above E. All of the above

Q10. Which of the following is a popular application used to inspect traffic that flows on a network?

A. SniffAndSnort B. Telnet C. nslookup D. tracert / traceroute E. Wireshark

Q11. OSPF uses which routing algorithm?

A. Bellman-Ford B. Dijkstra C. A* D. Floyd-Warshall E. Prim's algorithm

Q12. The counting-to-infinity problem is associated with:

A. Dijkstra and OSPF B. Bellman-Ford and RIP C. BGP only D. All routing protocols E. None of the above

Q13. BGP is an example of a(n):

A. IGP B. EGP C. Distance-vector protocol D. Link-state protocol E. Hybrid protocol

Long-Form & Calculation

Q14. Zone file compilation. [5]

Compile a zone file for the domain `xx.co.za`. Do not include the SOA record. Include only the resource records indicated:

- `xx.co.za` has a single computer at IP `41.203.18.59`.
- The web server can be accessed at `https://xx.co.za` and `https://www.xx.co.za`. The owner does not want multiple RRs to update if the IP changes.
- The primary mail server is `ASPMX.L.GOOGLE.COM` and the secondary mail server is `ALT1.ASPMX.L.GOOGLE.COM` (primary should be preferred).
- The primary name server is `ns1.host-h.net` and the secondary is `ns2.host-h.net`.
- There is a subdomain `abc.xx.co.za` using the same name servers.
- The IPv6 address of the single computer is `2001::1234`.
- The single computer is a Raspberry Pi.

Q15. Dijkstra's algorithm. [6]

Given the following network: A–B (cost 3), A–E (cost 2), B–G (cost 4), E–D (cost 2), E–F (cost 4), D–F (cost 3), D–C (cost 7), F–C (cost 8). Run Dijkstra's algorithm from node A. Show each iteration including the Done set, Candidates, cost to each node, and prior nodes.

Q16. nslookup trace. [4]

Describe the sequence of nslookup queries you would perform to iteratively resolve `www.example.com`, starting from a root name server at `192.5.5.241`. Show each server queried and what you ask for.

Memo / Answer Key

MCQ Answers

A1. B (Resolves domain names to IP addresses).

A2. B (Port 53).

A3. D (Both TCP and UDP — UDP for queries, TCP for zone transfers).

A4. E (`m.root-servers.net` — lexicographically last of a–m).

A5. D (More than one — the SNMP agent is both client and server).

A6. C (250).

A7. B (Individuals — `.nom.za` is for personal names).

- A8.** B (ccTLD — British Indian Ocean Territory).
- A9.** C (NS records — root servers only know where TLD name servers are).
- A10.** E (Wireshark).
- A11.** B (Dijkstra).
- A12.** B (Bellman-Ford and RIP).
- A13.** B (EGP — Exterior Gateway Protocol).

Long-Form Answers

A14. Zone file:

```
@ A 41.203.18.50
www CNAME xx.co.za.
@ MX 5 ASPMX.L.GOOGLE.COM.
@ MX 10 ALT1.ASPMX.L.GOOGLE.COM.
@ NS ns1.host-h.net.
@ NS ns2.host-h.net.
abc NS ns1.host-h.net.
abc NS ns2.host-h.net.
@ AAAA 2001::1234
@ HOST Raspberry Pi
```

Note: `www CNAME xx.co.za.` (with trailing dot) creates the alias without a separate A record, so only `@ A` needs updating if the IP changes.

A15. Dijkstra from A (network: A–B=3, A–E=2, B–G=4, E–D=2, E–F=4/E–D–F=2+3=5 so F via D, D–C=7, F–C=8):

#	Done	Candidates	Costs (B C D E F G)	Prior (B C D E F G)
1	A	E	3 – – 2 – –	E – – A – –
2	AE	BDF	3 – 4 2 6 –	E – A A E –
3	AEB	DFG	3 – 4 2 6 7	E – A A E B
4	AEBG	DF	3 – 4 2 6 7	E – A A E B
5	AEBGD	F	3 – 4 2 6 7	— keep F=6 via E
6	AEBGDF	C	3 14 4 2 6 7	— C via D (4+7=11) or F (6+8=14) → 11, prior=D

Final shortest paths from A: B=3(A), C=11(D), D=4(E), E=2(A), F=6(E), G=7(B).

A16. Iterative nslookup trace:

```
> server 192.5.5.241      ← point to f.root-servers.net
> set type=ns
> com.                    ← get NS records for .com → A-M.GTLD-SERVERS.NET
> server [one of the above, e.g. A.GTLD-SERVERS.NET]
> example.com.            ← get NS records for example.com → e.g.
ns1.example.com
> server [ns1.example.com address]
> set type=a
> www.example.com.        ← get A record → 192.0.32.10
```